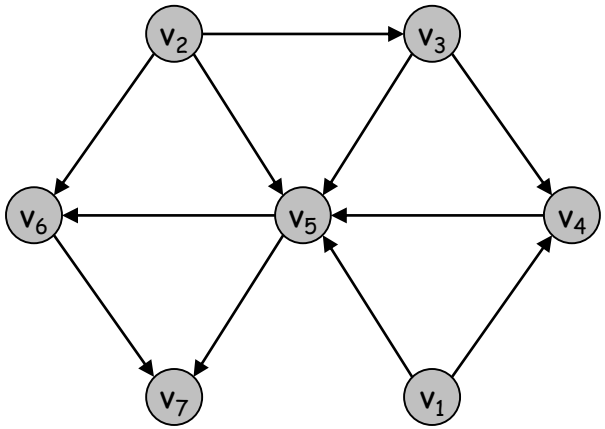
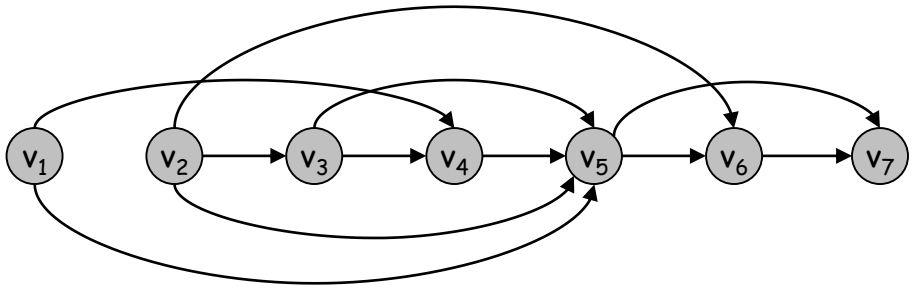
Directed Acyclic Graphs

A DAG is a directed graph that contains no directed cycles. DAG can be used for topological ordering.

A topological order of a directed graph $G = (V, E)$ is an ordering of its nodes as $v_1, v_2, \ldots, v_n$ so that for every edge (v_i, v_j) we have $i < j$.

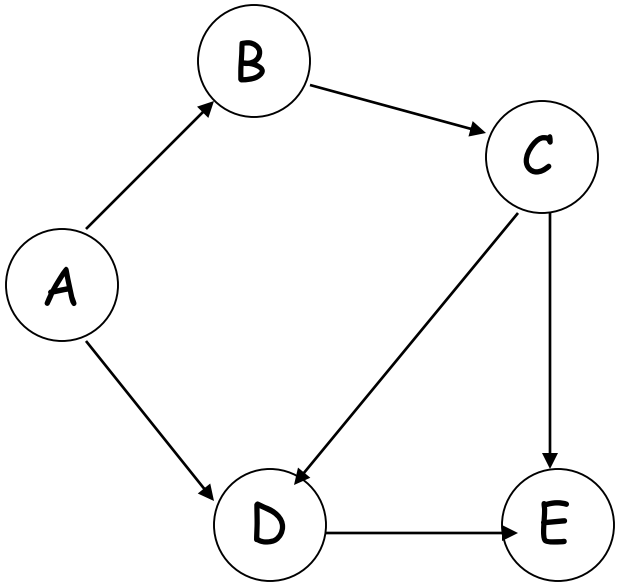


DAG



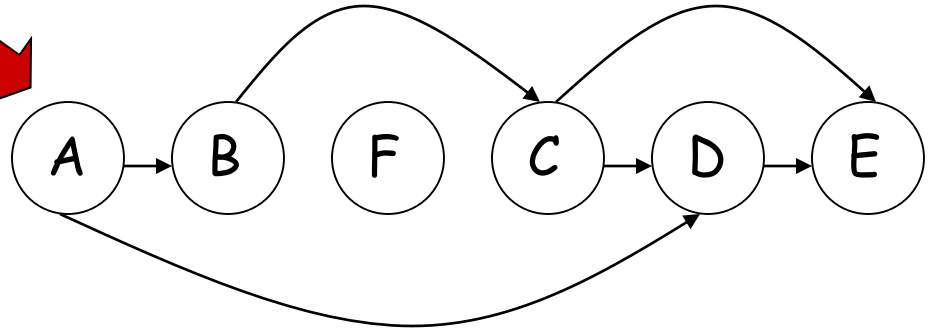
Topological ordering

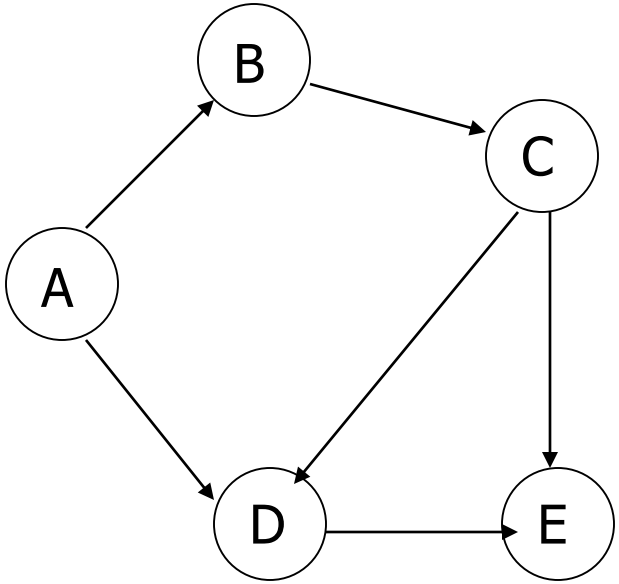
Another example



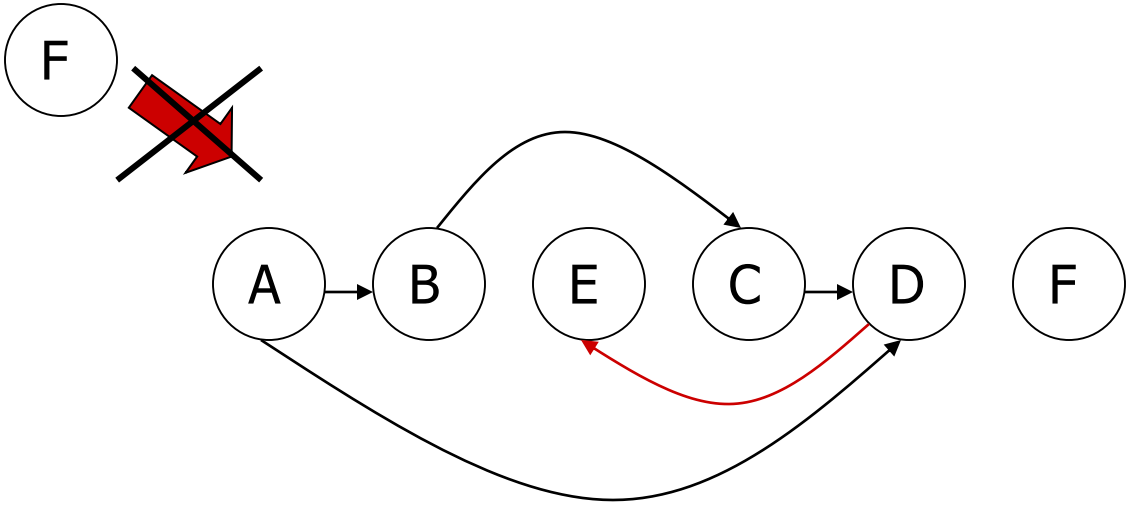
F

Any linear ordering in which all the arrows go to the right (same direction) is a valid solution.





Not a valid topological sort!

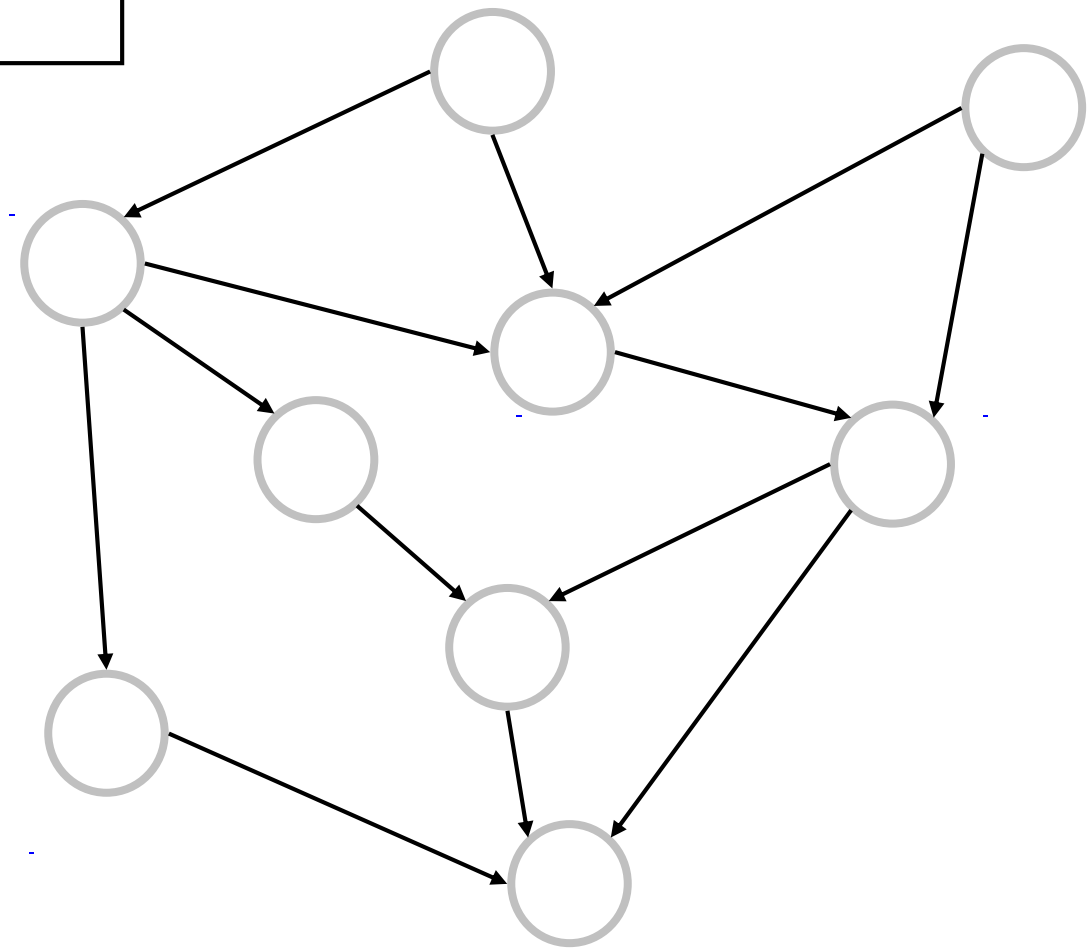


Algorithm for Topological Sorting

```
TopologicalSort(G)
    H ← G      // Temporary copy of G
    n ← G.numVertices()
    while H is not empty do
        Let v be a vertex with no outgoing edges
        Label v ← n
        n ← n - 1
        Remove v from H
```

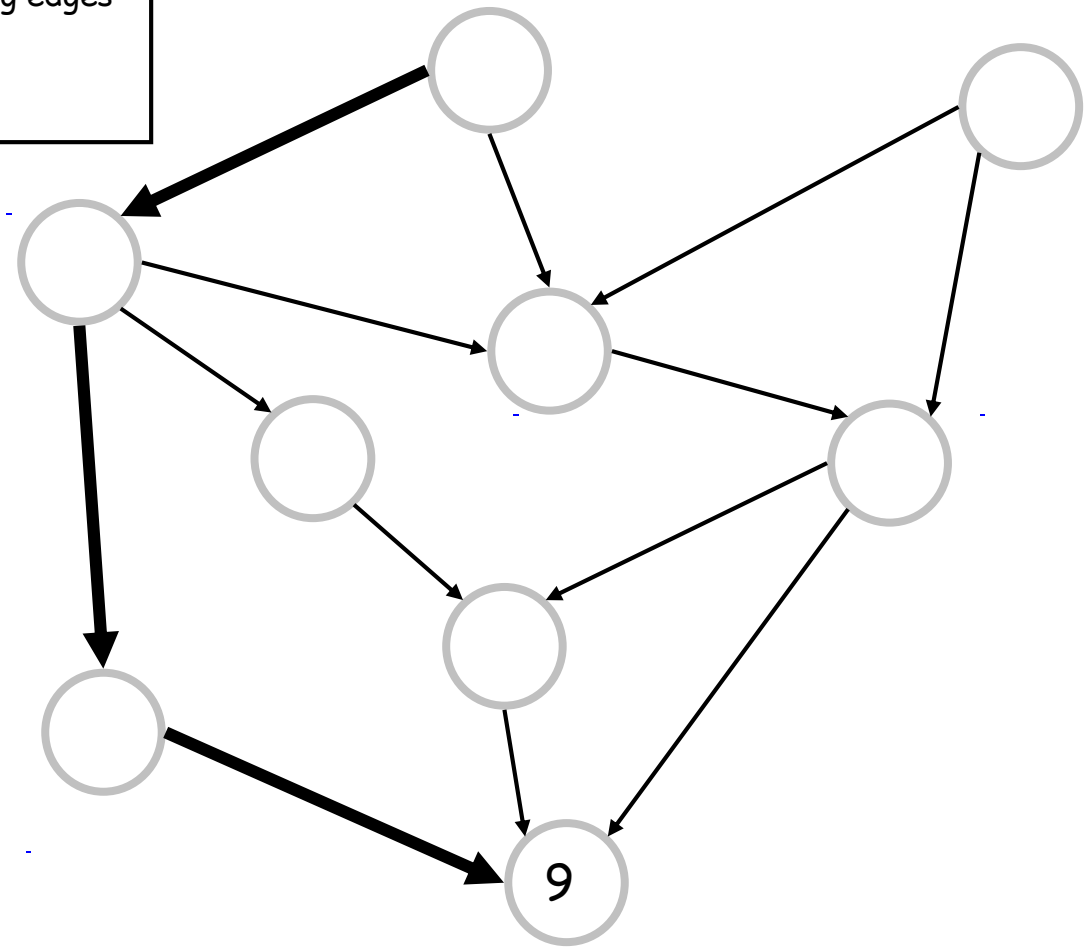
Topological Sorting Example

```
TopologicalSort(G)
    H ← G// Temporary copy of G
    n ← G.numVertices()
    while H is not empty do
        Let v  be a vertex with no outgoing edges
        Label v ← n
        n ← n - 1
        Remove v from H
```



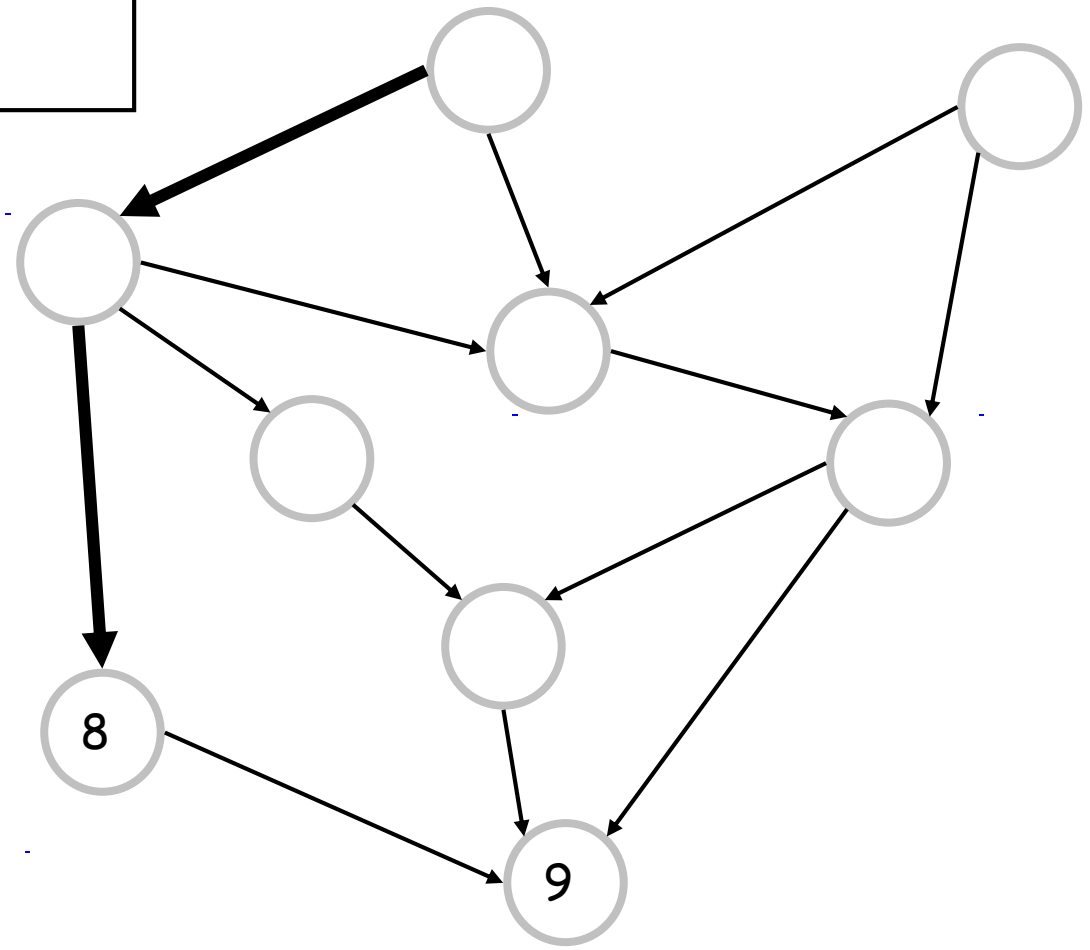
Topological Sorting Example

```
TopologicalSort(G)
    H ← G// Temporary copy of G
    n ← G.numVertices()
    while H is not empty do
        Let v be a vertex with no outgoing edges
        Label v ← n
        n ← n - 1
        Remove v from H
```



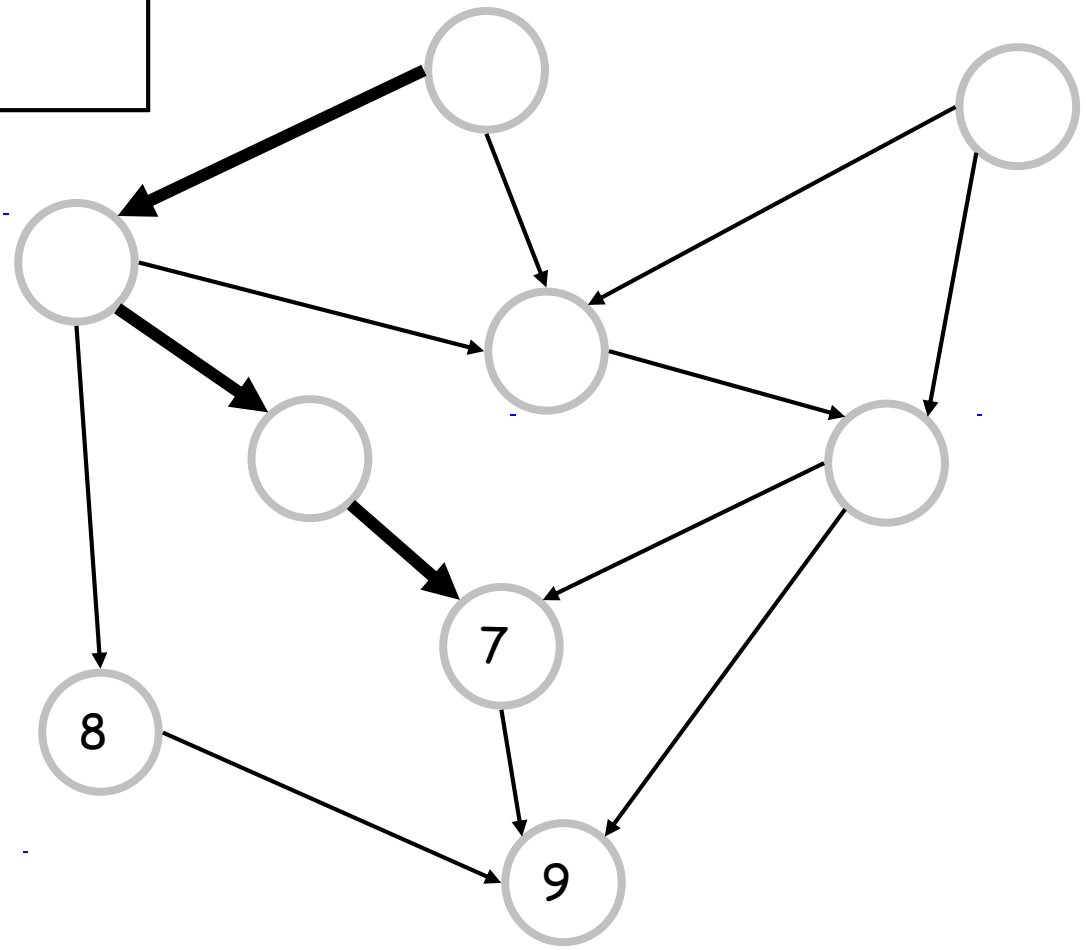
Topological Sorting Example

```
TopologicalSort(G)
    H ← G// Temporary copy of G
    n ← G.numVertices()
    while H is not empty do
        Let v be a vertex with no outgoing edges
        Label v ← n
        n ← n - 1
        Remove v from H
```



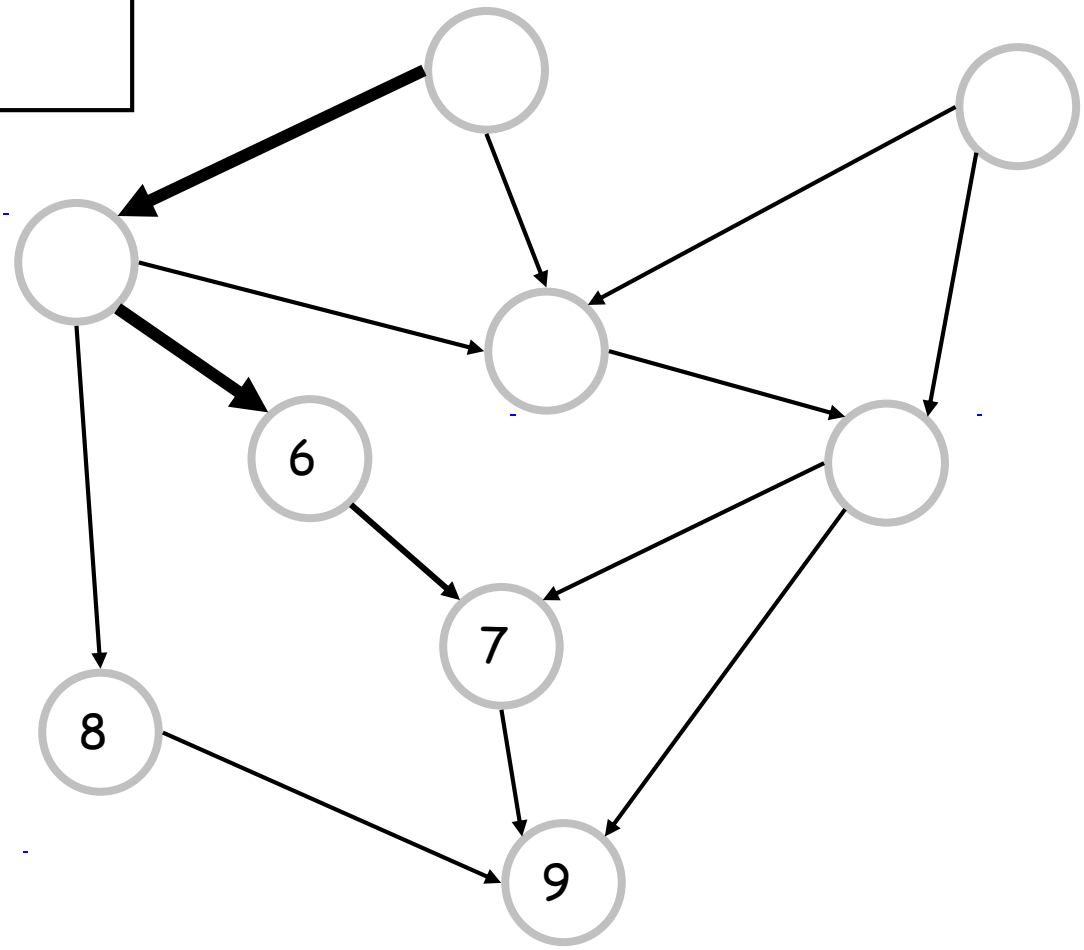
Topological Sorting Example

```
TopologicalSort(G)
    H ← G// Temporary copy of G
    n ← G.numVertices()
    while H is not empty do
        Let v be a vertex with no outgoing edges
        Label v ← n
        n ← n - 1
        Remove v from H
```



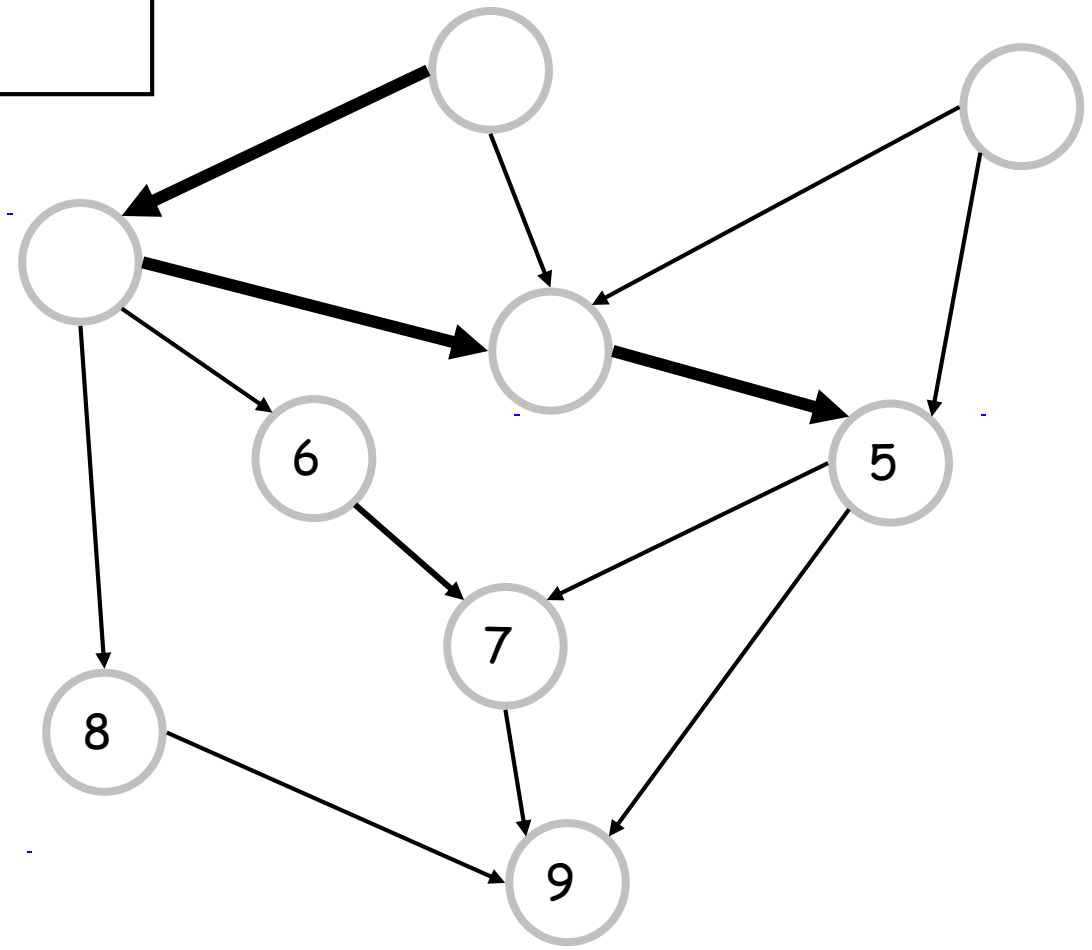
Topological Sorting Example

```
TopologicalSort(G)
    H ← G// Temporary copy of G
    n ← G.numVertices()
    while H is not empty do
        Let v be a vertex with no outgoing edges
        Label v ← n
        n ← n - 1
        Remove v from H
```



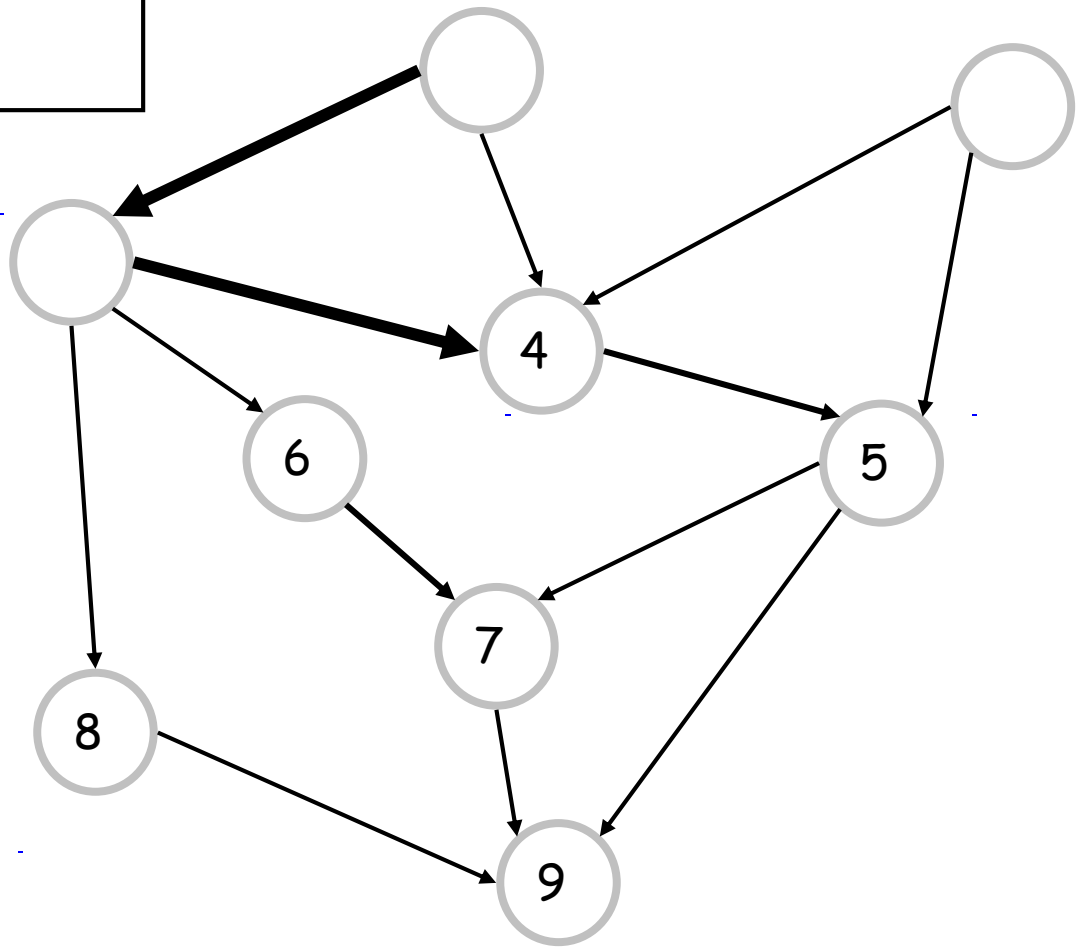
Topological Sorting Example

```
TopologicalSort(G)
    H ← G// Temporary copy of G
    n ← G.numVertices()
    while H is not empty do
        Let v be a vertex with no outgoing edges
        Label v ← n
        n ← n - 1
        Remove v from H
```



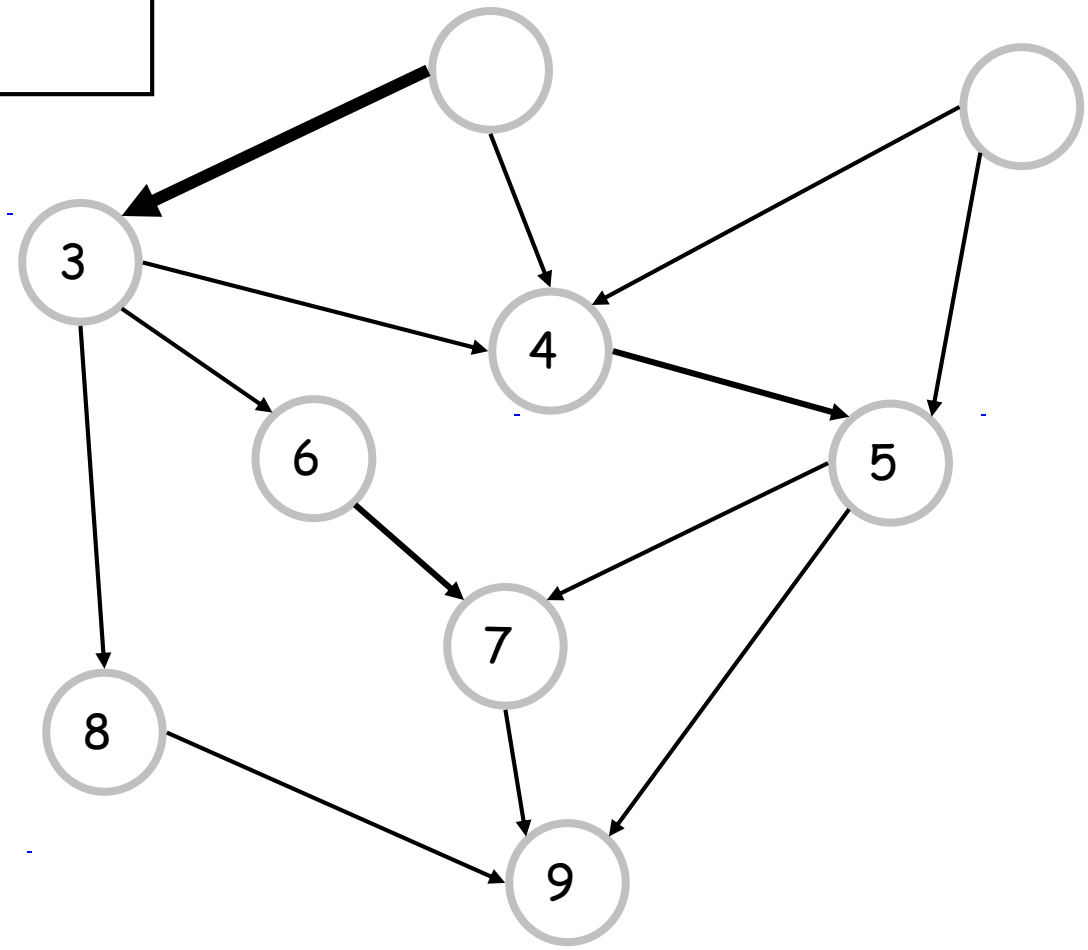
Topological Sorting Example

```
TopologicalSort(G)
    H ← G// Temporary copy of G
    n ← G.numVertices()
    while H is not empty do
        Let v be a vertex with no outgoing edges
        Label v ← n
        n ← n - 1
        Remove v from H
```



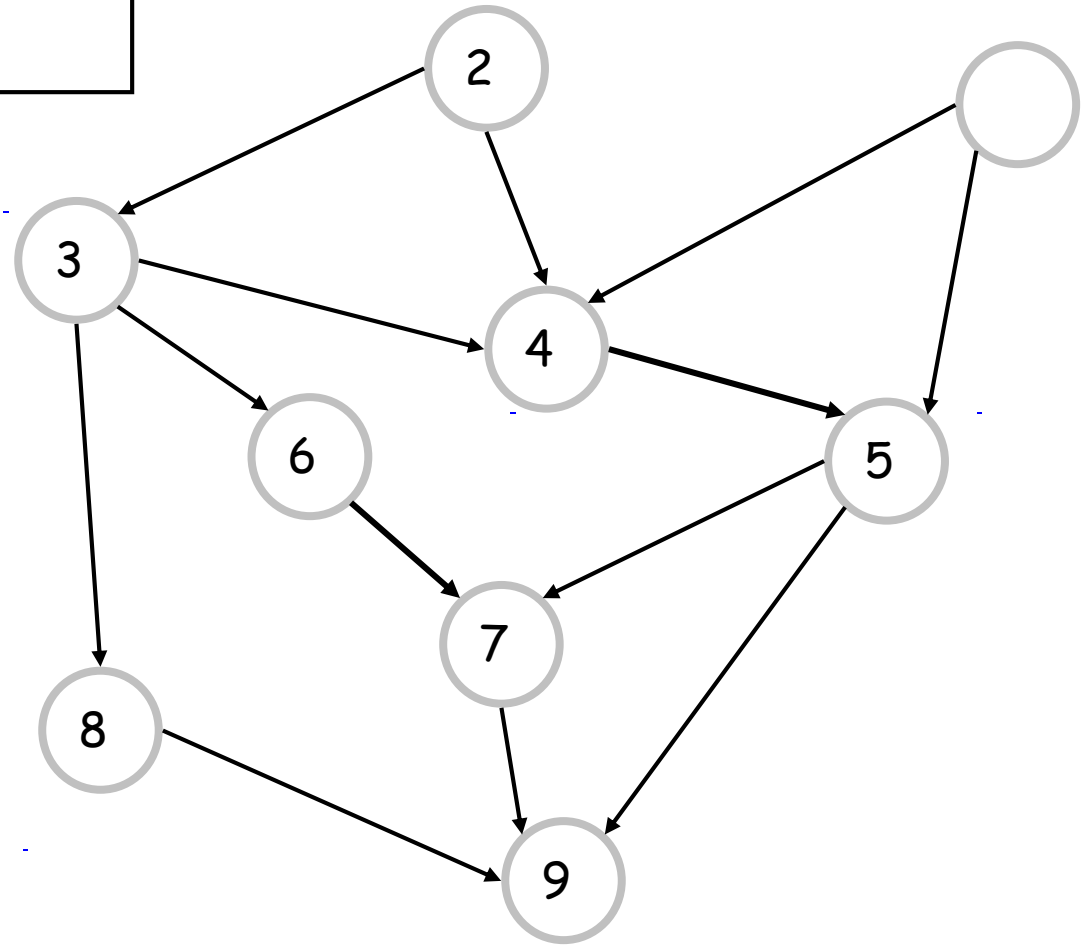
Topological Sorting Example

```
TopologicalSort(G)
    H ← G// Temporary copy of G
    n ← G.numVertices()
    while H is not empty do
        Let v be a vertex with no outgoing edges
        Label v ← n
        n ← n - 1
        Remove v from H
```



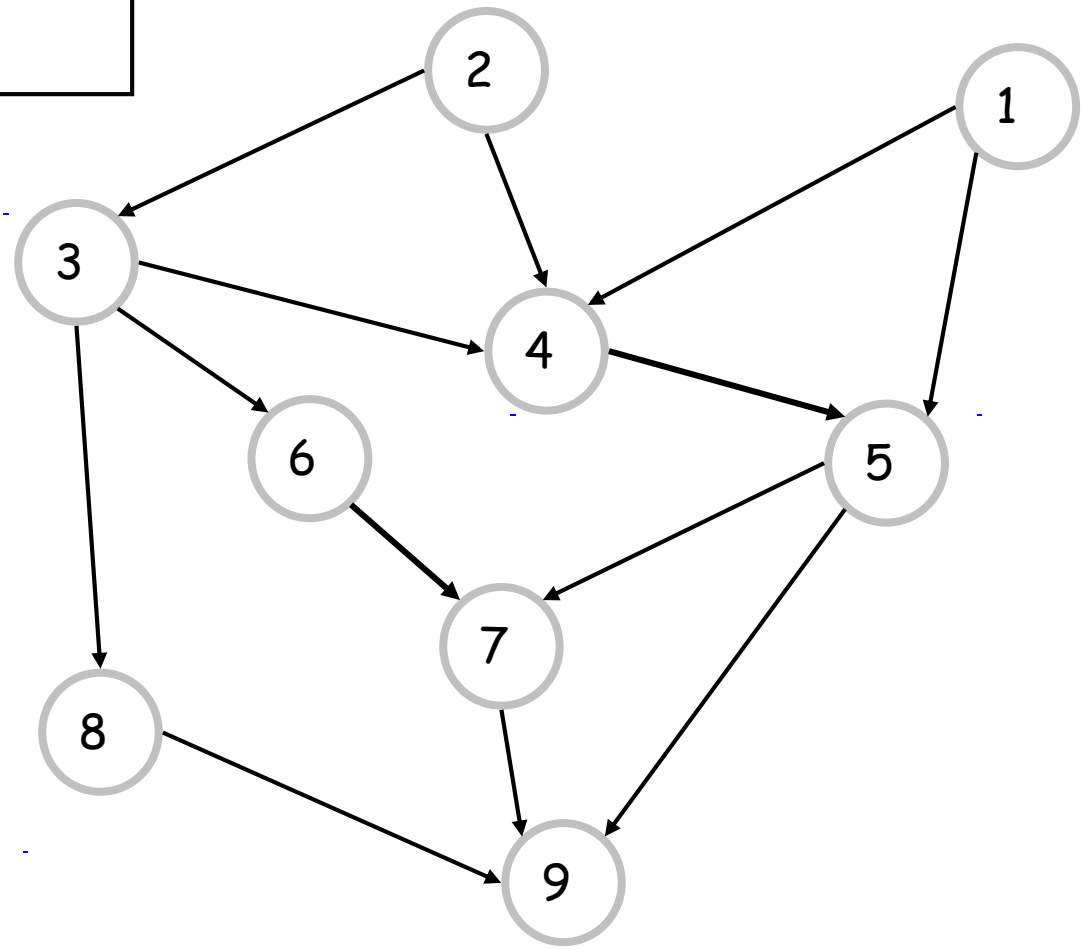
Topological Sorting Example

```
TopologicalSort(G)
    H ← G// Temporary copy of G
    n ← G.numVertices()
    while H is not empty do
        Let v be a vertex with no outgoing edges
        Label v ← n
        n ← n - 1
        Remove v from H
```



Topological Sorting Example

```
TopologicalSort(G)
    H ← G// Temporary copy of G
    n ← G.numVertices()
    while H is not empty do
        Let v be a vertex with no outgoing edges
        Label v ← n
        n ← n - 1
        Remove v from H
```



Topological Sorting Algorithm using DFS

Simulate the algorithm by using depth-first search

```
Algorithm topologicalDFS(G)
    Input DAG G
    Output topological ordering of G

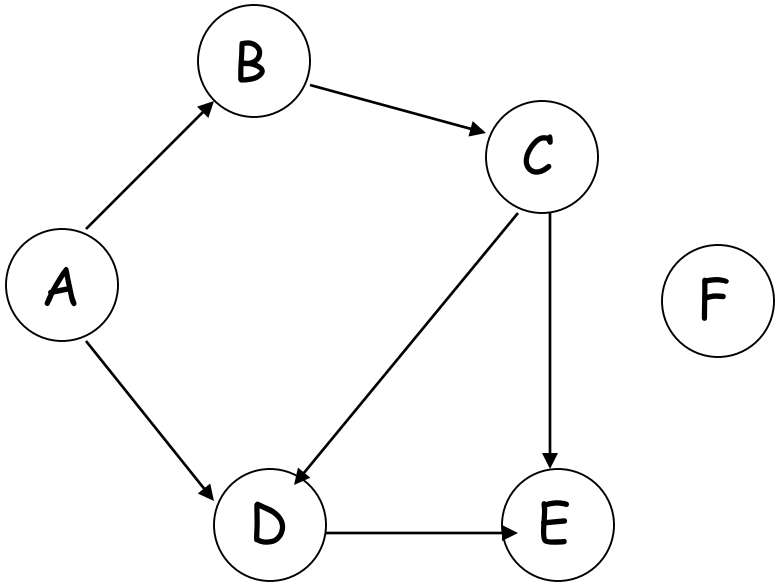
  n ← G.numVertices()
   for all  u ∈ G.vertices()
      setLabel(u, UNEXPLORED)
   for all  e ∈ G.edges()
      setLabel(e, UNEXPLORED)
   for all  v ∈ G.vertices()
     if  getLabel(v) = UNEXPLORED
         topologicalDFS(G, v)
```

```
Algorithm topologicalDFS(G, v)
    Input graph G and a start vertex v of G
    Output labeling of the vertices of G
      in the connected component of v

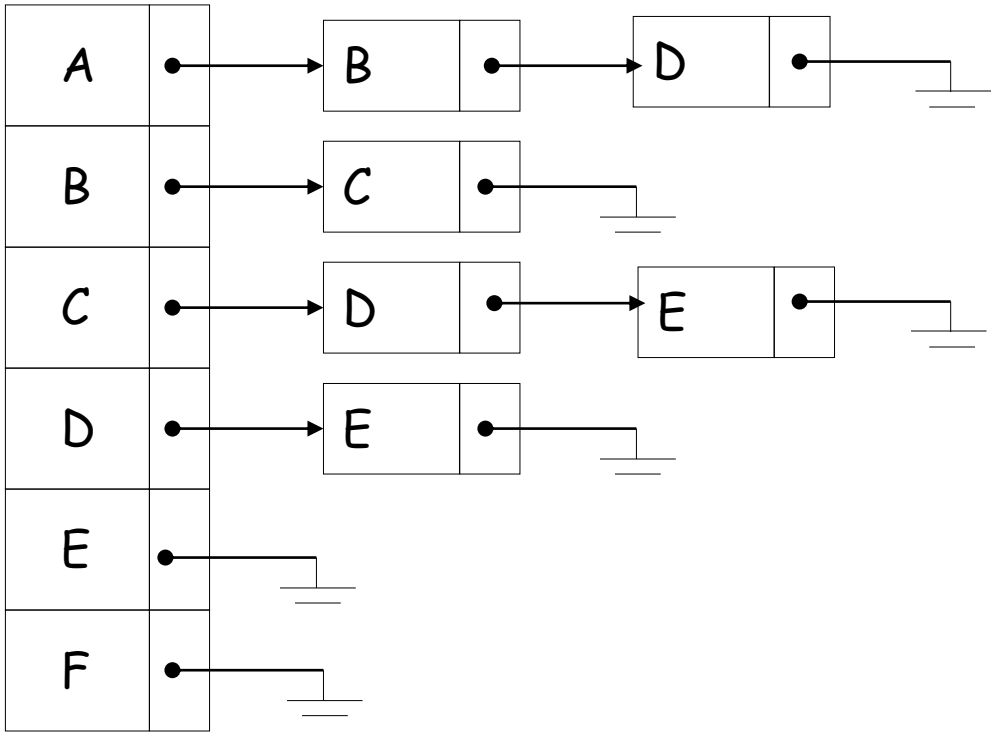
    setLabel(v, VISITED)
    for all  e ∈ G.incidentEdges(v)
       if  getLabel(e) = UNEXPLORED
          w ← connect(v,e)
          if getLabel(w) = UNEXPLORED
             setLabel(e, DISCOVERY)
             topologicalDFS(G, w)
          else
          Label v with topological number n
          n ← n - 1
```

Topological Sort Algorithm: Analysis

For input graph G = (V, E),
- Find a vertex with in-degree 0
- Remove its edges
- Place vertex into output

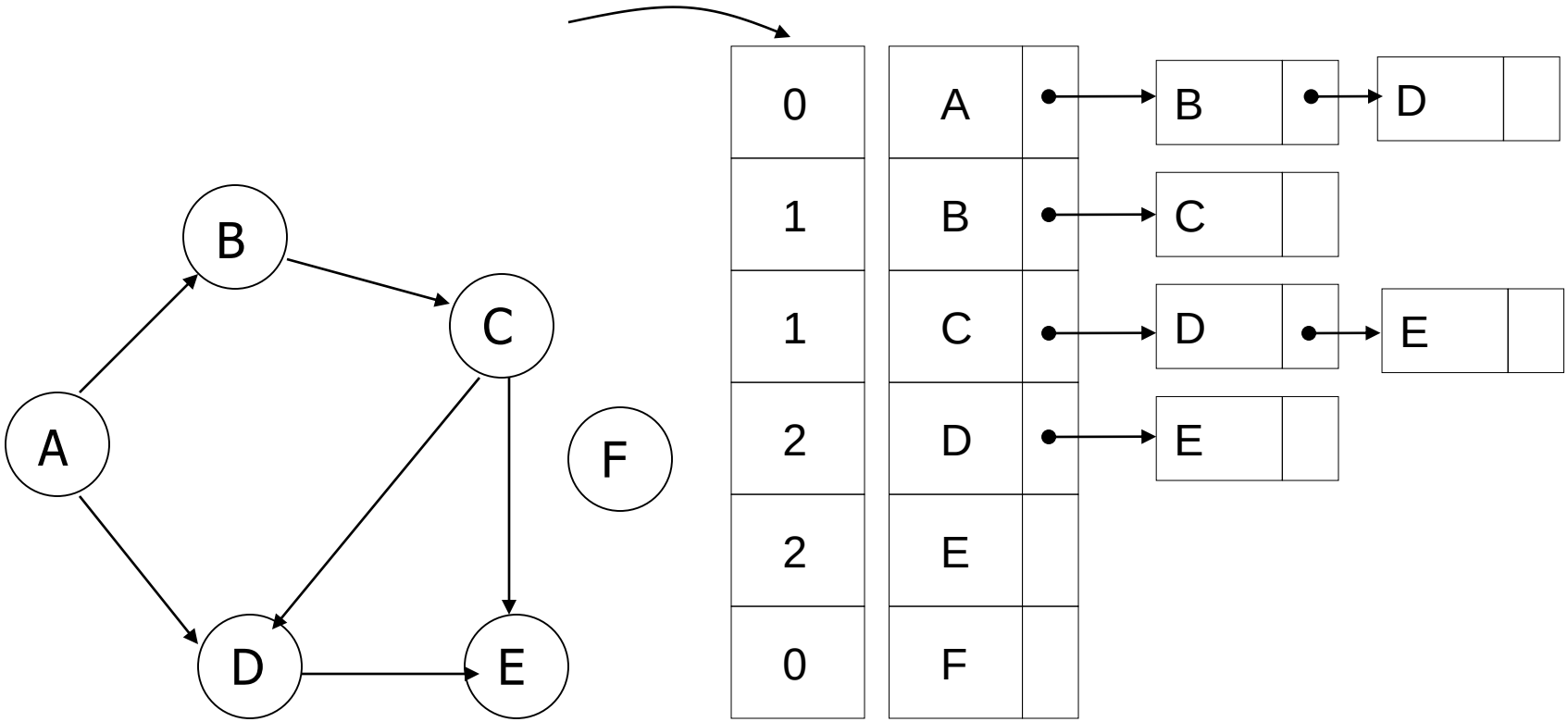


Adjacency list representation



Topological Sort: Analysis

Calculate and store In-Degree of all vertices in an array
- Find vertex with in-degree 0: Search this array
- Remove its edges: Update this array



Topological Sort: Analysis

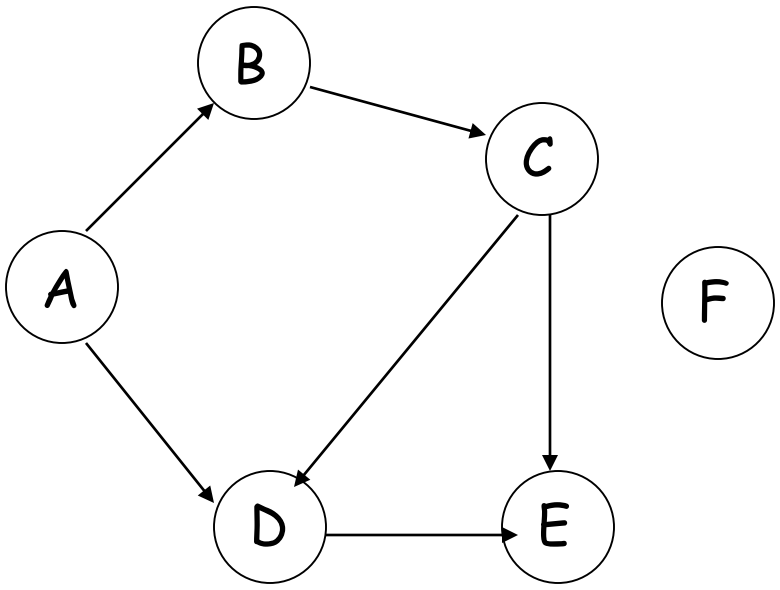
For input graph $G = (V, E)$,
Break down total time into:
- Find vertices with in-degree 0: $O(|V|^2)$
- Remove edges: $O(|E|)$
- Place vertices in output: $O(|V|)$

Total time = $O(|V|^2 + |E|)$

You can reduce the search time for in-degree

Topological Sort : Analysis (tightbound)

Initialize and maintain a queue of vertices with in-degree 0

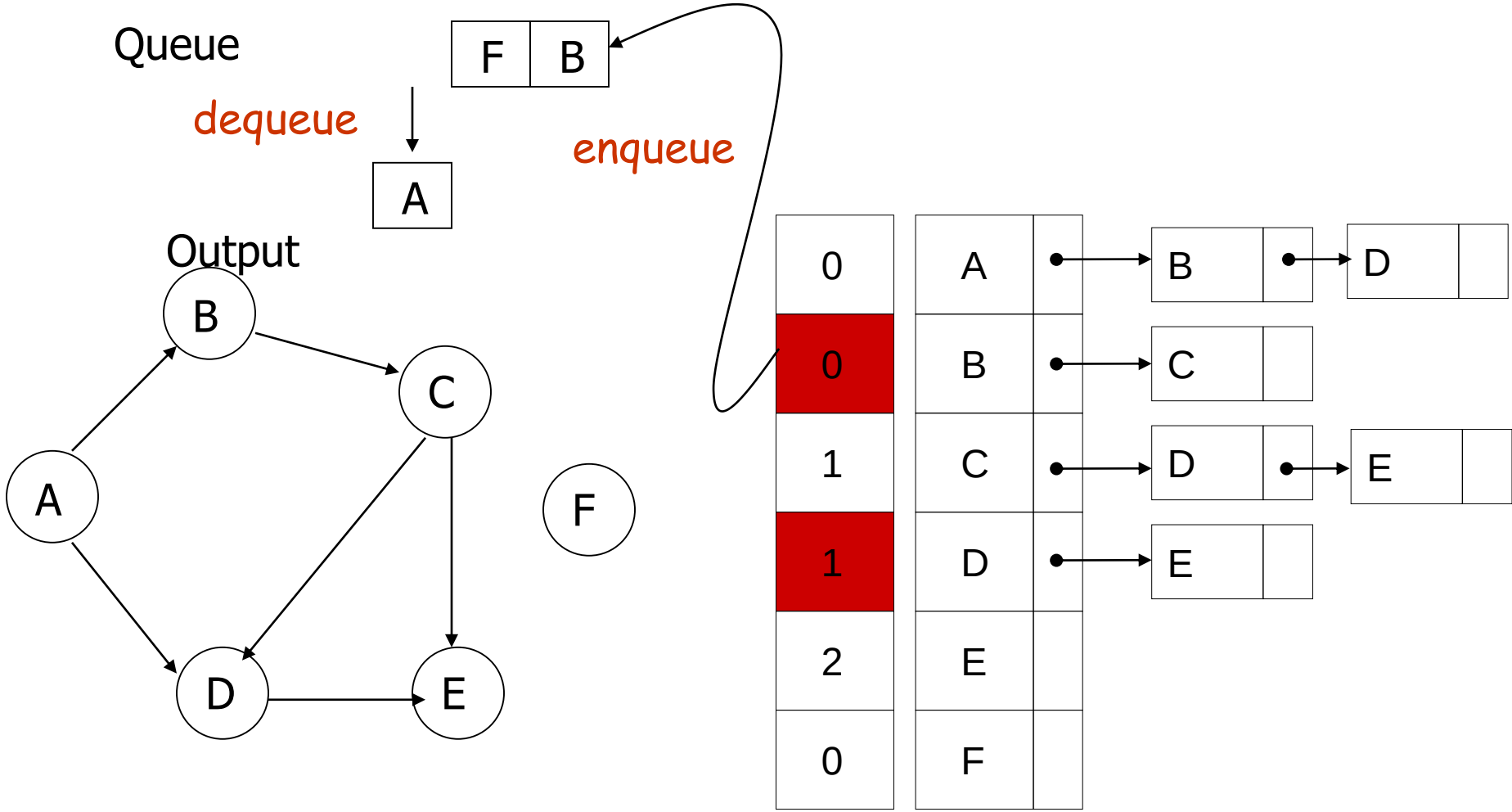


In-Degree array

0	A	B → D
1	B	C
1	C	D → E
2	D	E
2	E	
0	F	

Topological Sort : Analysis (tightbound)

After each vertex is output, when updating In-Degree array, enqueue any vertex whose In-Degree has become zero



Topological Sort Algorithm (tightbound)

Store each vertex's in-degree in an array
Initialize a queue with all in-degree zero vertices

While there are vertices remaining in the queue

- Dequeue and output a vertex
- Reduce in-degree of all vertices adjacent to it by 1
- Enqueue any of these vertices whose in-degree became zero

Topological Sort Algorithm (tightbound)

For input graph $G = (V, E)$,
Break down total time into

- Initialize in-degree array: $O(|E|)$
- Initialize Queue with in-degree 0 vertices: $O(|V|)$
- Dequeue and output vertex: $|V|$ vertices, each takes only $O(1)$ to dequeue and output. Total time = $O(|V|)$
- Reduce in-degree of all vertices adjacent to a vertex and Enqueue any in-degree 0 vertices: $O(|E|)$

Total time = $O(|V| + |E|)$